

TestNG

Test **Next Generation**



What · History · Why

Pros · Cons · Highlight

Features · Users · Reference

I

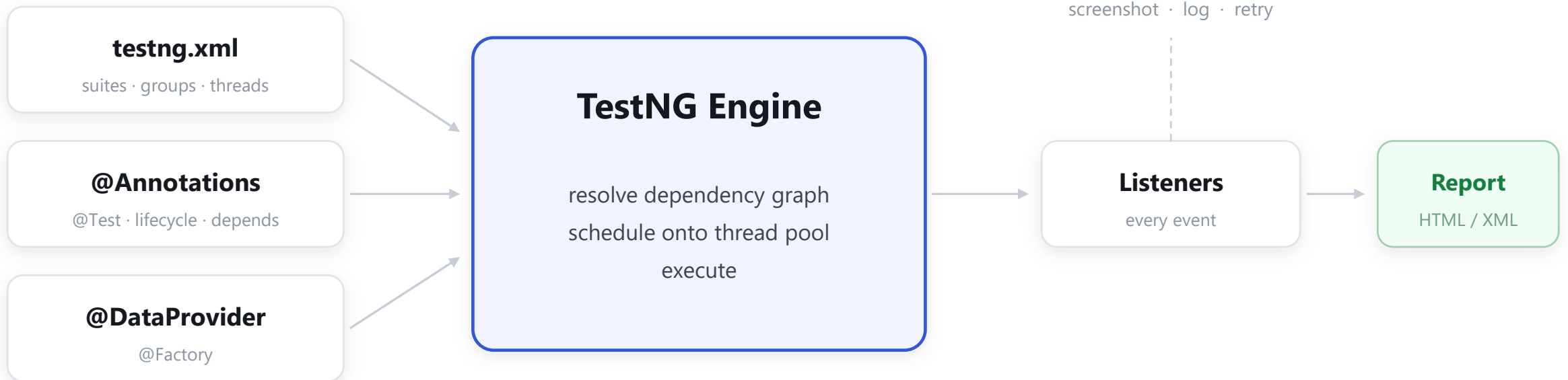
What · History · Why



Write · Organise · **Orchestrate**

Open-source testing framework for Java.

Config in — decisions out





2004

JUnit 3 ruled Java. And it **wasn't enough**.

The method name **was** the API

```
// JUnit 3  
public void testLogin() { ... }  
// drop four letters → the test silently disappears
```



Cédric Beust

Didn't patch JUnit. **Rewrote it.**

Inspired by JUnit & NUnit · built for integration testing

Three walls



No groups

Run only the smoke tests? No mechanism.



No dependencies

Login fails, checkout runs anyway — and fails too.



No parameters

Ten datasets meant ten copy-pasted methods.

JUnit 3 was built for isolated unit tests. Integration tests are not isolated.

II

Pros · Cons · Highlight



Four things you get



Order

Lifecycle · priority ·
dependsOn



Data

@DataProvider · @Factory



Speed

Native parallel execution



Control

testng.xml · listeners · CI

One real failure, not three

WITHOUT DEPENDENCIES

login
FAIL

addToCart
FAIL

checkout
FAIL

3 red reports · 1 cause

dependsOnMethods

login
FAIL

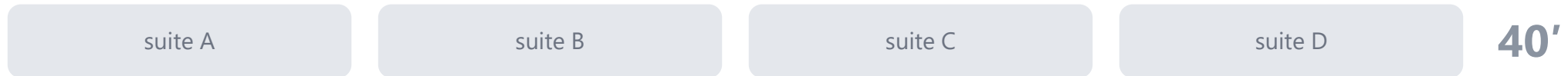
addToCart
SKIPPED

checkout
SKIPPED

1 red report · 1 cause

40 minutes → 10

SEQUENTIAL



PARALLEL · thread-count="4"



Granularity: methods · classes · tests · instances

What it costs



Complexity

Groups, dependencies, factories, listeners, XML. Powerful combined — and a real learning curve.

Misused dependencies hide the real failure behind a chain of skips.



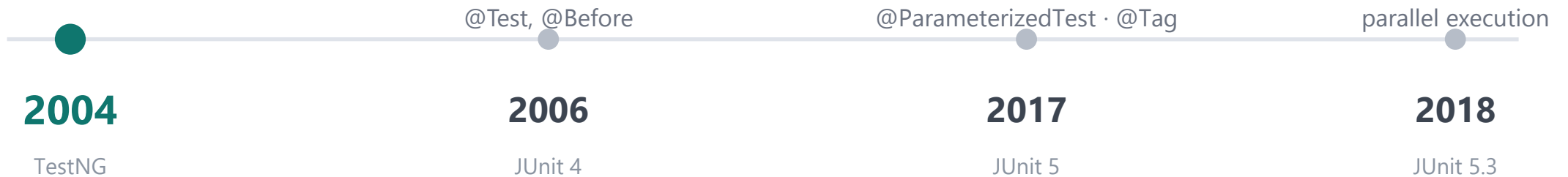
Overkill for unit tests

One monolithic dependency vs JUnit 5's modular Platform / Jupiter / Vintage.

If you only write fast isolated unit tests, the orchestration engine is dead weight — and Spring Boot defaults to JUnit 5.

The real win: getting copied

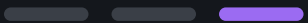
annotations · groups
dependencies · parallel



14 years for the ecosystem to catch up with the 1.0 feature list

III

Features · Users · Reference



Annotation-driven lifecycle

@BeforeSuite

@BeforeTest

@BeforeClass

@BeforeMethod

@Test

@AfterMethod

@AfterClass

@AfterTest

@AfterSuite

Groups

smoke · regression

@DataProvider

one test, many datasets

@Factory

many instances, one class

Listeners

screenshot · log · retry

Four lifecycle levels: suite → test → class → method

Change the run, not the code

```
<suite parallel="methods" thread-count="4">
```

```
<test name="Smoke">
```

```
<groups>    include: smoke, critical
```

```
<classes>   LoginTest, CartTest
```

Suite → Test → Class

Every commit → smoke only

Nightly → full regression

Same codebase.

Different XML file.

Wherever a suite gets big

Automation / QA engineers

Selenium & API suites — the dominant home of TestNG

Developers

Integration & service-level tests

DevOps

Gating pipelines by group



Runs from Maven, Gradle and the command line — so it drops straight into any pipeline

Still shipping **today**

MAVEN CENTRAL	V7.12.0	LAST COMMIT	YESTERDAY	STARS	2.1K
CONTRIBUTORS	218	LICENSE	APACHE-2.0	 JAVA	11+

testng.org · **github.com/testng-team/testng**

Live project figures, captured 21 September 2026

Not a test runner.
A test orchestrator.

Thank you.

